CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

CSPIT — Department of Computer Engineering · ITUE203 Web Development Frameworks

Student Name	Dhruv Bhoi	**Student ID**	**D26CE154**
Academic Year	2026–27 (ODD)	**Semester**	3rd Semester, B.Tech CE

Practical 6: Rendering External JSON Data using Fetch API, Search, and Filter

Practical Aim & Tools

Aim: Fetch and render external JSON datasets using the Fetch API, async/await, debounced search, category filters, dependent State-City dropdowns, sorting, and client-side pagination.

Tools & Technologies: JavaScript Fetch API, Promises, Async/Await, JSON Parsing, Debounce Utility

1. Architecture: JSON Data Rendering (`assets/js/data-render.js`)

StudentHub features two dynamic catalog modules backed by JSON files:
• Events Catalog (`events.php` / `data/events.json`): Fetches campus hackathons, seminars, and sports meets. Supports real-time text search, category filtering (Workshop, Technical, Cultural, Sports), date/title sorting, and pagination (6 items per page).
• Student Directory (`directory.php` / `data/students.json`): Fetches student directory records. Features a dependent State → City filter (selecting Gujarat dynamically populates Anand, Vadodara, Ahmedabad, Surat) along with course filters and sorting.

2. Implementation Code: Fetch API & Dependent Dropdowns

Listing: data-render.js — Async Fetch and Dependent Filter Logic

```
async function loadData(src) {
  statusEl.textContent = 'Loading records…';
  try {
    const res = await fetch(src);
    if (!res.ok) throw new Error('HTTP ' + res.status);
    allData = await res.json();
    statusEl.textContent = '';
    render();
  } catch (err) {
    statusEl.innerHTML = '<span class="error">Failed to load records.</span>';
  }
}

// Dependent State -> City Filter
const CITIES = {
  'Gujarat': ['Anand', 'Vadodara', 'Ahmedabad', 'Surat', 'Rajkot'],
```

```
  'Maharashtra': ['Mumbai', 'Pune', 'Nagpur', 'Nashik']
};
stateSelect?.addEventListener('change', () => {
  const st = stateSelect.value;
  citySelect.innerHTML = '<option value="">All cities</option>';
  if (CITIES[st]) {
    citySelect.disabled = false;
    CITIES[st].forEach(c => citySelect.add(new Option(c, c)));
  } else {
    citySelect.disabled = true;
  }
  render();
});
```

3. Verification Screenshot: Dynamic Events Catalog

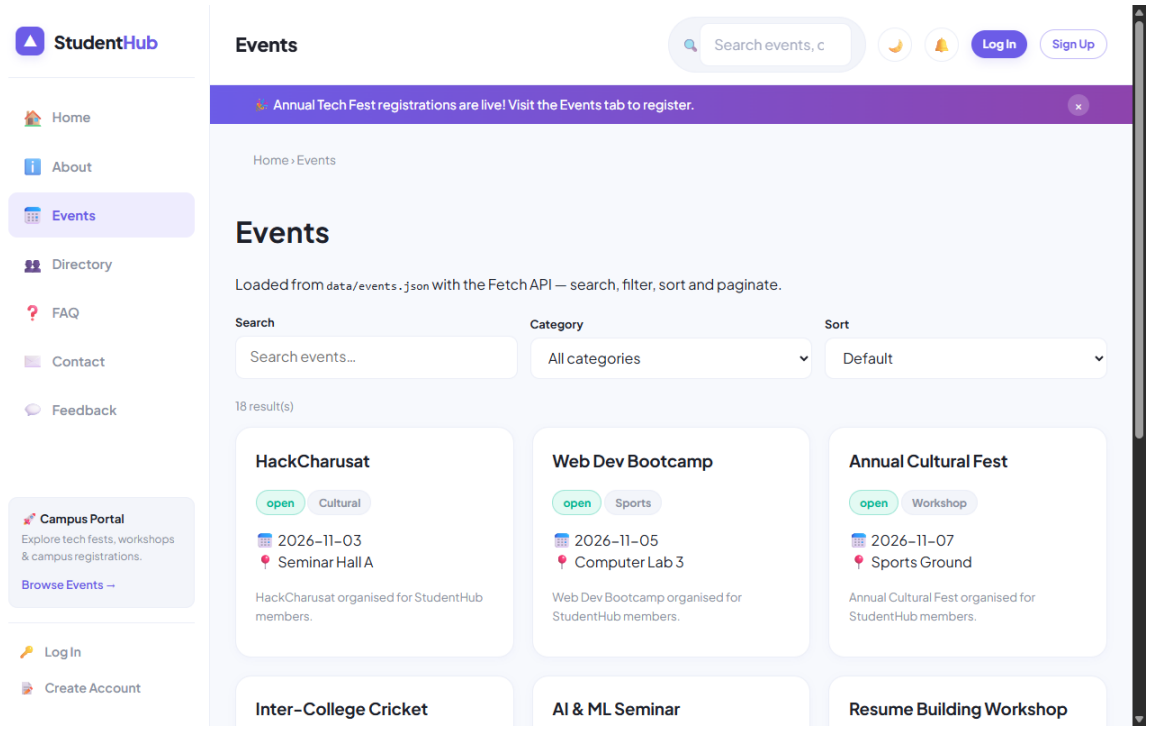


Figure: Events Catalog with Debounced Search, Category Filter, and Dynamic Cards Grid

4. Verification Screenshot: Student Directory with Dependent Dropdown

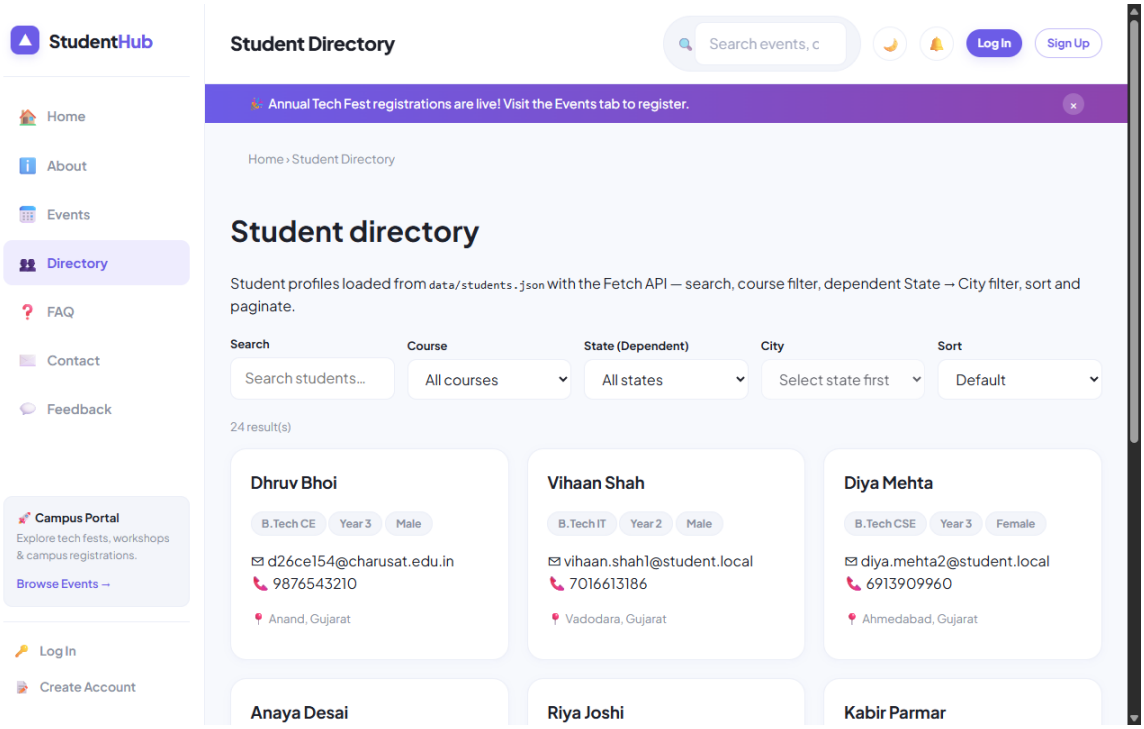


Figure: Student Directory with Dependent State -> City Filter and Top Student Dhruv Bhoi

5. Key Questions & Analysis

Q: How is JSON fetched, parsed, and rendered dynamically?

Ans: The browser calls `window.fetch(src)`, returns a Promise that resolves to a Response object, and parses the payload with `res.json()`. The resulting JavaScript array is transformed through `.filter()`, `.sort()`, and `.slice()` before building DOM elements via template literals.

Q: How are loading, error, and empty states handled?

Ans: A dedicated `<div id="dataStatus">` element with `aria-live="polite"` shows a spinner during network transport, displays user-friendly retry alerts if `!res.ok`, and shows an 'Empty records found' notice if the search query matches 0 records.

Practical 6 Conclusion & Learning Outcomes

Leveraged the Fetch API and Async/Await to fetch, parse, and render external JSON datasets for Events and the Student Directory. Implemented debounced text search, category filters, dependent State-City cascading dropdowns, multi-field sorting, and client-side pagination.